Real-Time Gaze Detection System

Faculty of Artificial Intelligence and Data Management
Course: Pattern Recognition

Team Members:
Sama Mohamed Tawfik
Zizi Mostafa Hamed
Shaza Abdulnaser Sayed
Sara Hassan Mostafa

Date: 03/05/2026

Table of content

Abstract

This project utilizes **OpenCV** and **MediaPipe** to create a real-time gaze monitoring system using webcam. The software identifies iris and eye corner landmarks to calculate how far a user's gaze deviates from the center of the screen. If the user looks away beyond defined horizontal or vertical thresholds for a specific duration, the system triggers an alert. This alert consists of a visual "FOCUS!" message, a red screen, and an audible beep generated by the **winsound** library. By measuring eye movement relative to the eye socket center, the tool serves as an automated way to track attention and discourage distraction

1. Introduction

Computer Vision is a field of technology that helps machines "see" and understand the world. A key part of this is **Object Detection**, which allows computers to find specific items ,like faces within an image. Usually, computers need to study a large **dataset** (a collection of thousands of pictures) to learn how to recognize these objects correctly. In this project, we apply these concepts to monitor a person's focus:

The Dataset: Instead of training our own model, we use **MediaPipe**, a powerful tool that has already learned from millions of images how to find 468 points on a human face.

Object Detection: Our script, uses this technology to treat the **iris** and **eyelids** as specific objects to be tracked.

Application: Once the computer detects the eyes, it uses math to see if the iris is staying in the center or moving away. If it detects the user looking away for too long, it triggers an alarm to help them refocus.

This turns complex AI technology into a simple tool for better productivity.

2. Dataset / Input Description

The Dataset

Pre-trained Knowledge: The project uses **MediaPipe**, which was trained on a huge dataset of real-world human faces.

Face Mesh Technology: This dataset allows the computer to recognize 468 landmarks on a person's face.

Eye Specifics: we used the part of the dataset that helps detect the iris and the eye lids.

Reliability/scalability: Because the dataset includes many different types of faces and lighting conditions, the system works well for almost anyone.

The Input

Live Video Feed: The primary input for is the live stream from a standard laptop webcam.

Real-Time Frames: The camera captures individual pictures (frames) every second and feeds them into the program.

Color Conversion: Since computers "see" colors and shapes differently than we do, the program quickly translates each image into a format it can understand.

Landmark Coordinates: Finally, the system turns the image of your face into a map of numbers. These numbers tell the computer exactly where your irises are located at every moment

3. Methodology

The methodology for this project explains how Try5.py transforms a raw video feed into a working focus alert system. It follows a simple three-step process: **Detection**, **Calculation**, and **Action**.

1 Detection: Mapping the Face

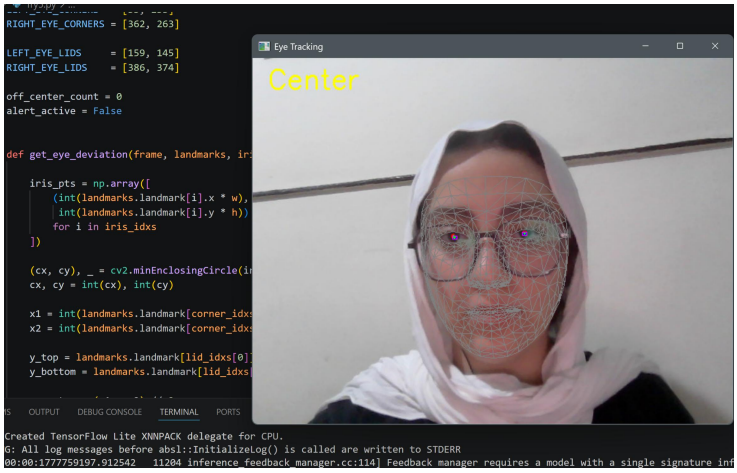
The first step is using the **MediaPipe Face Mesh** to find the user's face in the video frames.

The program uses a webcam to capture live video at a high speed.

It looks for specific "landmarks," which are numbered points on the face.

```
23   # ================== LANDMARKS ==================
24   LEFT_IRIS  = [468, 469, 470, 471, 472]
25   RIGHT_IRIS = [473, 474, 475, 476, 477]
26
27   LEFT_EYE_CORNERS  = [33, 133]
28   RIGHT_EYE_CORNERS = [362, 263]
29
30   LEFT_EYE_LIDS     = [159, 145]
31   RIGHT_EYE_LIDS    = [386, 374]
32
```

We focus on the points that mark the **iris** (the colored part of the eye) and the **corners** of the eyes.



2. Calculation: Measuring the Gaze

Once the points are found, the system uses math to see where the user is looking.

The program finds the exact **center** of the eye by looking at the corners and eyelids.

It then measures the distance between the **iris** and that center point.

If the iris moves too far to the left, right, up, or down, the computer calculates a "deviation" score.

We use a horizontal threshold (**X**) and a vertical threshold (**Y**) to decide if the movement is a distraction.

```
# -------- Decision --------
gaze_status = "Center"

if deviations:
    avg_dx = sum(d[0] for d in deviations) / len(deviations)
    avg_dy = sum(d[1] for d in deviations) / len(deviations)

    if abs(avg_dx) > THRESH_X or abs(avg_dy) > THRESH_Y:
        gaze_status = "Not Center"

    cv2.putText(frame, gaze_status, (20, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 1.2, (0, 255, 255), 2)
```

3. Action: The Alert System

The final step is to respond if the user is not paying attention.

The system uses a **frame counter** to make sure it doesn't beep for a simple blink.

If the user looks away for more than 20 frames, the script triggers an alert.

A **visual alert** appears as a red overlay with the word "FOCUS!" on the screen.

An **auditory alert** (a beep) is played using the winsound tool to grab the user's attention.

```
# -------- Alert logic --------
if gaze_status == "Not Center":
    off_center_count += 1
else:
    off_center_count = 0
    alert_active = False

if off_center_count > ALERT_FRAMES:
    overlay = frame.copy()
    cv2.rectangle(overlay, (0, 0), (w, h), (0, 0, 255), -1)
    cv2.addWeighted(overlay, 0.3, frame, 0.7, 0, frame)

    cv2.putText(frame, 'FOCUS!', (w // 3, h // 2),
                cv2.FONT_HERSHEY_SIMPLEX, 2.0, (255, 255, 255), 4)

    if not alert_active:
        winsound.Beep(1000, 300)
        alert_active = True
```

4. Experimental Work / Implementation

The Test Setup

To run the experiment, we used a laptop with a built-in webcam.

Environment: The test was done in a lit room to see how well the camera could pick up facial landmarks.

Tools: We made sure the **MediaPipe** and **OpenCV** libraries were correctly installed and that the system's sound was turned on for the audio alert.

Setting the Thresholds:

Before starting, we had to "tune" the sensitivity of the program.

Sensitivity: We tested different values for THRESH_X (horizontal) and THRESH_Y (vertical) to find a balance where the system wouldn't be too annoying but still catch when the user looked away.

Timing: We set the ALERT_FRAMES to 20, which the user has to look away for about a second before the alarm goes off. This prevents the alarm from triggering every time someone blinks.

Running the Experiment:

During the test, we performed three main actions to see how the software reacted:
A)Focusing: While looking directly at the screen, the system displayed "Center" and stayed quiet.

B)Brief Look Away: We looked away for just a split second. The system noticed the movement but did not trigger the alarm because of the 20-frame buffer.

C)Sustained Distraction: We looked away for several seconds. The program successfully detected the "Not Center" status, flashed the red "FOCUS!" warning, and played a beep sound.

Observations:

We observed that the system is very responsive. The use of the **Face Mesh** landmark points (like 468 for the iris) allowed the computer to track eyes succefully. The winsound library worked perfectly on Windows to provide that final nudge to get back to work

5. Results

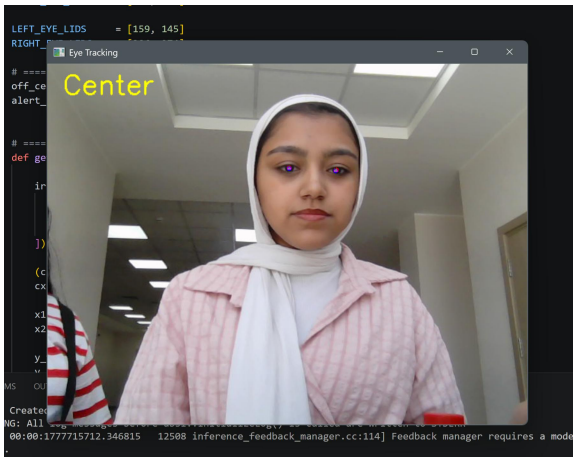
Accurate Tracking: The system successfully utilized **MediaPipe** landmarks to locate the exact location of the left and right irises in every video frame.

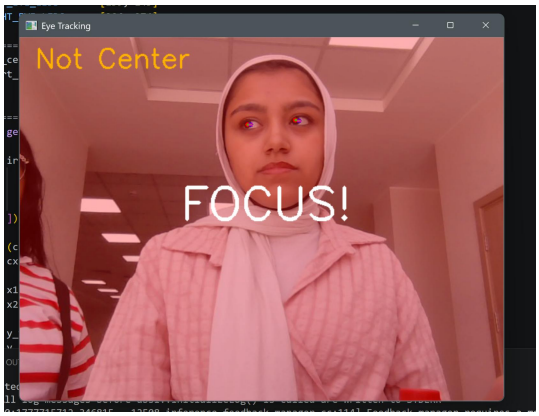
Smart Delay System: The use of an ALERT_FRAMES counter proved successful in ignoring natural blinks, ensuring that the alarm only triggered during sustained distraction.

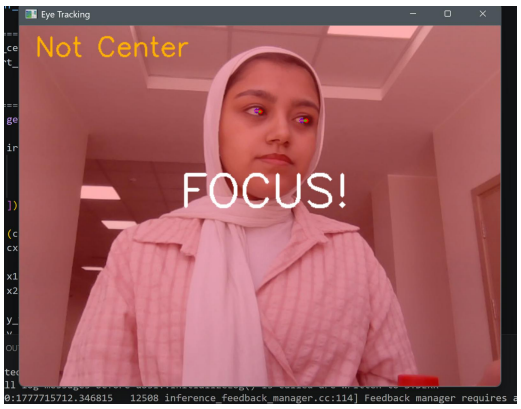
Effective Visual Alerts: When a distraction was detected, the screen successfully displayed a semi-transparent red overlay and a bold "FOCUS!" message to grab the user's attention.

Reliable Audio Feedback: The “winsound” library consistently played as soon as the alert state became active.

Real-Time Performance: The software processed the video feed smoothly, providing instant feedback on the user's gaze status without any noticeable lag







6. Discussion

The discussion section explores why the results are important and how the system actually behaves in different situations

Accuracy of Tracking: Using **MediaPipe** proved to be a great choice because it can find eye landmarks even if the user moves their head or sits in different lighting. This makes the system more reliable than older methods that only looked for simple shapes.

The Importance of Thresholds: A key part of the project was balancing the THRESH_X and THRESH_Y values. If these numbers are too small, the alarm goes off for every tiny movement; if they are too large, the system misses actual distractions. This shows that computer vision tools often need "fine-tuning" to fit the user's needs.

Handling Natural Movements: One of the biggest challenges in eye tracking is dealing with blinking. By using the ALERT_FRAMES counter, we successfully taught the computer to ignore quick blinks and only focus on long-term looks away.

User Feedback: The combination of a visual "FOCUS!" message and an audible beep creates a "multi-sensory" alert. This is more effective than just text alone because it uses both sight and sound to grab a distracted person's attention.

Potential Improvements: While the system works well on Windows using **winsound**, future versions could be updated to work on other computers or even use a phone's camera for better portability.

7. Conclusion

The goal of this project was to create a smart tool that helps people stay focused by monitoring their eye movements in real-time. By combining **Computer Vision** with a pre-trained **Dataset**, we successfully built a system that can tell the difference between working and being distracted.

Project Success: The script proved that a standard webcam is all you need to track a person's gaze accurately.

Effective Detection: Using **MediaPipe** allowed the computer to focus on specific landmarks, like the iris, to calculate exactly where a user is looking.

Practical Alert System: The project successfully used a mix of red visual warnings and sound beeps to pull a distracted user's attention back to their screen.

Smart Logic: By adding a frame counter, we ensured the system remains helpful without being annoying, as it ignores natural actions like blinking.

The project shows the potential of technology to enhance our everyday routines. It provides a solid basis for future solutions that can support workers and students in maintaining high levels of productivity in a distracted society.

8. Future Work

In the future, we can improve the system with these three simple ideas:

Works on All Computers: We can change the sound settings so the program works on Mac and Linux laptops, not just Windows.

Health Alerts: The system could be updated to track how much you blink and suggest a quick break if your eyes look tired.

Phone Version: We could turn the code into a mobile app so you can stay focused while studying with physical books instead of just a computer.